

Towards GPU-accelerated reconciliation

Enzo Marsot

April 2026

1 The current ALE algorithm

1.1 Description

Let S be a rooted species tree. We write its number of nodes as $|S|$. Let Γ be a distribution of clades. For each clade, we denote the set of its splits by $\text{bip}(\gamma)$.

Let e be a branch of S . Let $\bar{E}_e = \sum_{f \in \text{rec}(e)} p_{e \rightarrow f}^T E_f$. As in [ALERax], we have the following recursion equations :

$$E_e = S(E)_e + D(E)_e + T(E)_e + L(E)_e$$

Where

$$S(E)_e = p_e^S E_f E_g \text{ if } e \text{ is an internal branch with children } f \text{ and } g, \text{ otherwise } 0$$

$$D(E)_e = p_e^D E_e^2$$

$$T(E)_e = E_e \bar{E}_e$$

$$L(E)_e = p_e^L$$

$$\bar{E}_e = \sum_{f \in \text{rec}(e)} p_{e \rightarrow f}^T E_f$$

$$\Pi_{\gamma,e} = S_\gamma(\Pi)_e + D_\gamma(\Pi)_e + T_\gamma(\Pi)_e + SL_\gamma(\Pi)_e + DL_\gamma(\Pi)_e + TL_\gamma(\Pi)_e$$

Where

$$S_\gamma(\Pi)_e = p_e^S \sum_{(\gamma', \gamma'') \in \mathcal{B}(\gamma)} p(\gamma', \gamma'' \mid \gamma) (\Pi_{\gamma', f(e)} \Pi_{\gamma'', g(e)} + \Pi_{\gamma', g(e)} \Pi_{\gamma'', f(e)}) \quad (1)$$

$$D_\gamma(\Pi)_e = p_e^D \sum_{(\gamma', \gamma'') \in \mathcal{B}(\gamma)} p(\gamma', \gamma'' \mid \gamma) \Pi_{\gamma', e} \Pi_{\gamma'', e} \quad (2)$$

$$T_\gamma(\Pi)_e = \sum_{(\gamma', \gamma'') \in \mathcal{B}(\gamma)} p(\gamma', \gamma'' \mid \gamma) (\Pi_{\gamma', e} \bar{\Pi}_{\gamma'', e} + \Pi_{\gamma'', e} \bar{\Pi}_{\gamma', e}) \quad (3)$$

$$SL_\gamma(\Pi)_e = p_e^S (\Pi_{\gamma, f(e)} E_{g(e)} + \Pi_{\gamma, g(e)} E_{f(e)}) \quad (4)$$

$$DL_\gamma(\Pi)_e = 2p_e^D \Pi_{\gamma, e} E_e \quad (5)$$

$$TL_\gamma(\Pi)_e = \Pi_{\gamma, e} \bar{E}_e + E_e \bar{\Pi}_{\gamma, e} \quad (6)$$

$$\bar{\Pi}_{\gamma, e} = \sum_{f \in \text{rec}(e)} p_{e \rightarrow f}^T \Pi_{\gamma, f} \quad (7)$$

Remark 1. *The transfer probability is contained in the $\overline{E}$ term and the $\overline{\Pi}$ term.*

The fixed point equation for E can be solved by iterating the map $E \mapsto S(E) + D(E) + T(E) + L(E)$ until convergence, starting from a 0 vector and the converged value of E . Similarly, the fixed point equation for Π can be solved by iterating the map $\Pi \mapsto S_\gamma(\Pi) + D_\gamma(\Pi) + T_\gamma(\Pi) + SL_\gamma(\Pi) + DL_\gamma(\Pi) + TL_\gamma(\Pi)$ until convergence, starting from a 0 vector and the converged value of E . We prove convergence and contraction properties of these maps in the appendix.

1.2 Limitations of the current model

1.2.1 The ALE Undated model

No branch length utilization

- Even if imprecise or blurry, branch lengths for both the species trees and the gene trees are informative.

No gene family interdependence

- Events that affect several gene families at once are not explicitly modeled.

A model of evolution that is hard to extend

- Both the ALE Dated and Undated models rest on the assumption that daughter lineages of a given gene copy evolve independently. I do not know if this assumption can have a large impact on the inferred gene tree probabilities : in the article by Larget (2013), the authors say they construct a distribution of gene trees by MCMC using the "HKY85" model of evolution, which assumes independent evolution between gene copies. They then compare the simple relative frequencies of gene trees from the sample, and the probabilities obtained using CCPs and obtain good agreement. However, this does not help us know how important the independence assumption is. Efficiently computing the likelihood of a gene tree under the ALE models **requires this independence assumption**.

No modelling of extinct lineages The ALE dated model incorporates extinct lineages, as an outgroup branch, and the ALE undated model does not model extinct lineages at all. Using a single extinct lineage is a crude approximation : we can show that under a perfect molecular clock, it is feasible to reconstruct the complete species tree, which is much more informative than a outgroup representing all extinct lineages together. Integrating extinct lineages into a reconciliation model would require a complicated extension of the current models in order to simultaneously leverage species tree branch lengths, uncertain gene tree branch lengths, and use this information to construct the unobserved part of the species tree. After a discussion with Nicolas Lartillot, optimizing over this set of complete species trees seems technically difficult, and I don't know whether it is possible or if I can do it.

2 Optimization opportunities

We can start by noticing that the fixed-point map F_γ decomposes as a sum of terms which are fixed and obtained by the previous iterations, and self-dependent terms. Obtaining the S, D, T terms therefore amounts to gather/scatter operations which will be done only once per clade, while the SL, DL, TL terms will need to be recomputed for each iteration.

2.1 Parallelization opportunities

I have observed that there exist many opportunities for parallelization in the ALE undated algorithm (I am not certain about the undated model, but I expect it to be similar).

ALE Undated Let us focus on the Π matrix. This matrix has shape $[G, S]$ where G is the total number of clades in all gene families, and S is the number of species tree branches. We notice that the computation of $\Pi_{\gamma,\cdot}$ only depends on $\Pi_{\gamma,\cdot}$ and $\Pi_{\delta,\cdot}$ for subclades δ of γ . This means that the computation of Π has to be scheduled in waves that follow the DAG defined by the splits $\gamma \rightarrow (\gamma', \gamma'')$. The different gene families each give one DAG, and we can schedule clades from different gene families in the same wave. We can set a wave size that maximizes GPU occupancy, and not have to worry about underutilization : mixing different DAGs together ensures this will rarely happen.

The ALE algorithm is often used to reconcile a large number of gene families with a given species tree.

Optimization opportunities in ALE Dated

2.2 Computing the fixed strict-subclade terms

For each clade γ , the terms S_γ , D_γ , and T_γ only depend on strict subclades of γ . They should therefore be computed once, before the fixed-point loop for $\Pi_{\gamma,\cdot}$, and stored in the constant vector A_γ . Assuming that $\bar{\Pi}$ has already been computed for every strict subclade, the best implementation is a fused segmented reduction over the CCP splits of γ .

Let a split record $r \in \text{bip}(\gamma)$ contain

$$\ell_r, \quad r_r, \quad q_r = p(\ell_r, r_r \mid \gamma),$$

where ℓ_r and r_r are the two daughter clades. For every species branch e , compute

$$A_{\gamma,e} = \sum_{r \in \text{bip}(\gamma)} q_r [p_e^D \Pi_{\ell_r,e} \Pi_{r_r,e} + \Pi_{\ell_r,e} \bar{\Pi}_{r_r,e} + \Pi_{r_r,e} \bar{\Pi}_{\ell_r,e} + \mathbf{1}_{\{e \text{ internal}\}} p_e^S (\Pi_{\ell_r,f(e)} \Pi_{r_r,g(e)} + \Pi_{\ell_r,g(e)} \Pi_{r_r,f(e)})].$$

This writes the full fixed contribution

$$A_\gamma = S_\gamma + D_\gamma + T_\gamma$$

without materializing separate S , D , and T buffers.

In a GPU implementation, the natural custom kernel owns one output pair (γ, e) , or one small cooperative group owns it when the number of splits is large. The kernel loops over the split segment of γ , gathers the two daughter clades and their CCP weight, accumulates the D , T , and S contributions, and writes exactly one value $A_{\gamma,e}$:

```
acc = 0
for split in splits[gamma]:
    l, r, q = split.left, split.right, split.weight
    acc += q * pD[e] * Pi[l,e] * Pi[r,e]
    acc += q * (Pi[l,e] * Pibar[r,e] + Pi[r,e] * Pibar[l,e])
    if e is internal:
        f, g = child1[e], child2[e]
        acc += q * pS[e] * (Pi[l,f] * Pi[r,g] + Pi[l,g] * Pi[r,f])
A[gamma,e] = acc.
```

For this kernel, clade-major storage,

$$\Pi, \bar{\Pi}, A \in \mathbb{R}^{|\mathcal{G}| \times B},$$

is attractive because a warp processing consecutive branches for a fixed split reads contiguous rows $\Pi_{\ell_r,\cdot}$, $\Pi_{r_r,\cdot}$, $\bar{\Pi}_{\ell_r,\cdot}$, and $\bar{\Pi}_{r_r,\cdot}$. The S term still performs indirect reads at $f(e)$ and $g(e)$, but those are only two branch-index gathers per branch. The important point is to avoid forming a temporary tensor of shape **(parents, splits, branches)**; that tensor would multiply memory traffic and would likely dominate the runtime.

The kernel granularity should depend on the split distribution. If most clades have few splits, one thread per (γ, e) is simple and avoids synchronization. If some clades have many splits, a warp or block can cooperate on one (γ, e) and reduce over the split segment. If branch count is small or moderate, a block per γ with branch tiling is also viable. In all cases, this is conceptually one fused kernel for $S + D + T$ once $\bar{\Pi}$ is available.

2.3 Global and Specieswise modes

The expensive part of each fixed-point iteration is the same-clade linear operator

$$SL_\gamma(\Pi) + DL_\gamma(\Pi) + TL_\gamma(\Pi).$$

In the global and specieswise modes, the transfer probability only depends on the donor branch e , not on the receiver branch h . We can therefore write

$$p_{e \rightarrow h}^T = \alpha_e = \frac{p_e^T}{|\text{rec}(e)|} \quad \text{for } h \in \text{rec}(e).$$

For a fixed clade γ ,

$$\bar{\Pi}_{\gamma,e} = \alpha_e \sum_{h \in \text{rec}(e)} \Pi_{\gamma,h} = \alpha_e \left(\sum_{h \in \text{Br}(S)} \Pi_{\gamma,h} - \sum_{h \notin \text{rec}(e)} \Pi_{\gamma,h} \right).$$

Thus the dense-looking receiver sum is a rank-one operation plus a sparse correction over the branches that are not legal receivers.

Let $B = |\text{Br}(S)|$ and let $\mathcal{C}$ be a batch of clades whose strict subclades have already been computed. We use the branch-major convention

$$\Pi \in \mathbb{R}^{B \times |\mathcal{C}|}, \quad \Pi_{e,\gamma} = \Pi_{\gamma,e}.$$

With this convention, left multiplication by a branch matrix is written $M\Pi$. The column sum vector is

$$s = \mathbf{1}_B^\top \Pi \in \mathbb{R}^{1 \times |\mathcal{C}|},$$

which corresponds to `Pi.sum(dim=0)` in PyTorch. If the implementation stores the transpose, i.e. clade-major tensors of shape `(clades, branches)`, then the same operation appears as `Pi.sum(dim=1)` and the sparse multiplication is written $\Pi M^\top$ instead.

The same-clade update for the whole batch can be written

$$\Pi^{(k+1)} = A_{\mathcal{C}} + Vs + M\Pi^{(k)}.$$

Here $A_{\mathcal{C}}$ contains the terms that do not depend on the current clade values being iterated : the strict-subclade contributions from S_γ , D_γ , T_γ , and the terminal leaf-boundary contributions. It is computed once per clade batch, before the fixed-point loop. The vector $V \in \mathbb{R}^B$ is

$$V_e = E_e \alpha_e,$$

and Vs denotes the outer product $(Vs)_{e,\gamma} = V_e s_\gamma$. This is the vectorized part of $E_e \bar{\Pi}_{\gamma,e}$ that uses the sum over all branches.

The sparse correction matrix $M \in \mathbb{R}^{B \times B}$ contains all same-clade terms except the global receiver sum. Its entries are accumulated as follows :

$$\begin{aligned} M_{e,e} &+= 2p_e^D E_e + \bar{E}_e, \\ M_{e,f(e)} &+= p_e^S E_{g(e)} \quad \text{for internal branches } e, \\ M_{e,g(e)} &+= p_e^S E_{f(e)} \quad \text{for internal branches } e, \\ M_{e,h} &+= -E_e \alpha_e \quad \text{for every } h \notin \text{rec}(e). \end{aligned}$$

The first line is the sum of the DL term and the $\Pi_{\gamma,e}\bar{E}_e$ part of TL . The next two lines are the SL child contributions. The last line subtracts the illegal receivers that were included in the global branch sum s_γ . If several rules contribute to the same entry, the coefficients are simply added. Since the illegal receiver set is typically the ancestors of e plus any convention-specific excluded branches, this correction is sparse.

Equation (2.3) is useful because V and M depend only on the species tree, the DTL parameters, and the already-computed extinction vector E , not on γ . They can therefore be reused for all clades and all fixed-point iterations until the parameters change.

Kernel count. With standard GPU primitives, the update can be implemented in three kernels : one reduction for $s = \mathbf{1}^\top \Pi$, one sparse-dense matrix multiplication for $M\Pi$, and one fused elementwise kernel computing $A_C + Vs + M\Pi$. With a custom sparse kernel whose epilogue adds A_C and Vs , this becomes two kernels : reduction, then fused SpMM plus epilogue. A single kernel is possible only if each clade column can be handled by one cooperative block or persistent work unit that first computes its branch sum and then applies the sparse rows of M ; this is plausible when B is small enough, but it is less compatible with generic cuSPARSE-style SpMM because the column sum requires a synchronization before the row updates.

2.4 Pairwise mode

3 Auto-differentiation for efficient optimization

3.1 The analytic formulas : implicit differentiation through the fixed point solution

3.2 Computing the gradient in practice

3.2.1 VJP iteration

3.2.2 Unrolling

3.2.3 Scheduling gradient computations

We can compute gradients corresponding to groups of gene trees at once, and accumulate gradients.

4 Scheduling the computations on a GPU

5 Computing the different terms

Explain here what the difficulties are for computing the different terms : memory-intensive terms, hard to coalesce memory accesses, etc.

5.1 Scheduling waves

5.1.1 Forward

The set of clades forms a DAG, defined by the splits $\gamma \rightarrow (\gamma', \gamma'')$. We can schedule computations in waves, where each wave corresponds to a set of clades which can be computed in parallel. Our goal is to maximize GPU occupancy, in order to hide the latency of launching kernels and accessing memory. So we want to maximize wave size, under the constraint that all the necessary data fits in GPU memory.

The list of operations we will need to perform is as follows :

- Compute E . This will not be a problem, we can do it in parallel over branches and gene families.
 - Compute Π . We consider n gene families, which we will process in parallel.
- At any time, the GPU will have to store the following data :
- The

5.1.2 Backward

6 Numerics

6.1 Floating point formats

On CUDA GPUs, it is easiest to work with the float32 format. We can use even lower precision formats such as float16 or bfloat16, but we will have to be careful about numerical stability when doing so. It is easy to start parameter optimization in some format, then continue in another format. bfloat16 is probably better for our use case. For the early stages of optimization, we can accept the lower precision of bfloat16, but we cannot accept underflow or overflow, which is likely to happen in float16.

6.2 GPU resources

The GPU has the following resources :

- SFU (Special Function Units) : used for exp, log, etc.
- CUDA cores : used for arithmetic operations (e.g. addition, multiplication)
- Tensor cores : used for matrix multiplications, in either TF32, float16, or bfloat16 formats.

The tensor cores have the highest throughput, especially on modern CUDA GPUs. We would like to use them as much as possible, but it seems only the transfer term can be written as a true matrix multiplication, and it only seems suitable to express it as a dense matrix multiplication in the pairwise mode. We will therefore have to rely on CUDA cores, and carefully managing memory accesses.

6.3 Working in log-space

As is frequent with the sum-product algorithm, the probabilities we are computing can become very small. Furthermore, working with high precision formats on GPUs (above fp64) is complicated and inefficient. We will therefore work in log space, and use the $\log \circ \sum \circ \exp$ sequence of operations to compute sums. We will have to experimentally verify that this does not cause too much numerical instability. Anecdotally, my initial experiments show that the fp32 causes instability only at the very end of optimization, where we can switch to fp64.

7 Methodology

Our goal is to minimize the runtime of the reconciliation likelihood optimization. To do that, the following steps will be taken :

- Minimize the runtime of the forward pass
- Minimize the runtime of the backward pass
- Hide the latency of memory-intensive operations.

7.1 Approximations and optimizations

- Instead of updating parameters using exact gradients, we can use stochastic gradient descent, by only computing the gradient on a subset of gene families : this is mini-batch SGD. We can use LBFGS to do that, and use a schedule for the batch size.
- We can try different optimizers, such as pseudo-Newton methods, plain SGD, Adam, etc. This will be **more easily done if we make the likelihood and gradient computation PyTorch-compatible**. To make the likelihood computation PyTorch-compatible, we can create custom backward and forward functions, specifying some subset of gene families and other parameters such as wave size. We will pass the parameters as PyTorch tensors, such as the transfer matrix, or the dtl parameters. We will therefore be able to easily use any PyTorch optimizer, and do our testing within a simple Jupyter notebook.

7.2 Tools

- We will use regular PyTorch code as a baseline for the forward and backward passes. Since we are working in log-space, it is essential to cleanly handle $-\infty$ values (which inevitably appear as representants of impossible events), so that we will often need to write custom backward functions.
- Production code will either use CUDA kernels, or use an open-source GPU library, or some DSL such as Triton/Gluon/CuTe DSL.

8 Experimental results

8.1 Forward time

8.2 Backward time

8.3 Is the GPU worth it ?

A Proof of convergence

In this proof we use the notation of AleRaxSupp.pdf : $\bar{x}_e$ denotes the average of x_h over receiver branches $h \in \text{rec}(e)$, and the scalar transfer probability p_e^T is written outside the barred term. Equivalently, one may absorb $p_e^T/|\text{rec}(e)|$ into the coefficients $p_{e \rightarrow h}^T$ used above.

Hypothèse 1 (Interior event probabilities and complete observation). *We assume $p_l^{\text{obs}} = 1$ for every species-tree leaf l , and*

$$p_e^S, p_e^D, p_e^T, p_e^L > 0$$

for every species-tree branch e . The strict positivity of p_e^S is the key condition for the argument below ; the full interior assumption is sufficient but stronger than necessary.

Let F be the extinction fixed-point map defined by equations (12)–(17) of the supplement, and let E be its fixed point. Define the survival vector

$$w_e = 1 - E_e.$$

Because $p_l^{\text{obs}} = 1$, terminal leaves have $E_l = 0$. Since $p_e^S > 0$, every terminal branch has positive survival probability. Inducting upward over the species tree gives

$$w_e > 0$$

for every branch e .

Define

$$L := DF(E).$$

For an internal branch e with children f, g ,

$$(Lu)_e = p_e^S(E_g u_f + E_f u_g) + 2p_e^D E_e u_e + p_e^T(\bar{E}_e u_e + E_e \bar{u}_e).$$

For a terminal branch e ,

$$(Lu)_e = 2p_e^D E_e u_e + p_e^T(\bar{E}_e u_e + E_e \bar{u}_e).$$

We now compare Lw with w . For an internal branch e ,

$$w_e - (Lw)_e = p_e^S w_f w_g + p_e^D w_e^2 + p_e^T w_e \bar{w}_e > 0.$$

For a terminal branch e ,

$$w_e - (Lw)_e = p_e^S + p_e^D w_e^2 + p_e^T w_e \bar{w}_e > 0.$$

Thus

$$Lw < w$$

componentwise.

Define the weighted sup norm

$$\|u\|_{w,\infty} := \max_e \frac{|u_e|}{w_e}.$$

Since L has nonnegative coefficients,

$$\|Lu\|_{w,\infty} \leq c \|u\|_{w,\infty}, \quad c := \max_e \frac{(Lw)_e}{w_e} < 1.$$

Therefore

$$\|DF(E)\|_{w,\infty} < 1.$$

Since the extinction map F is polynomial, $DF(x)$ depends continuously on x . Hence there exists a neighbourhood U of E and a constant $r < 1$ such that

$$\|DF(x)\|_{w,\infty} \leq r \quad \text{for all } x \in U.$$

Then for all $x, y \in U$, the mean-value formula gives

$$F(x) - F(y) = \int_0^1 DF(y + t(x - y))(x - y) dt,$$

and therefore

$$\|F(x) - F(y)\|_{w,\infty} \leq r \|x - y\|_{w,\infty}.$$

Thus the extinction fixed-point map is contracting in a neighbourhood of E .

Moreover, if U is chosen as a small closed ball around E , then it is stable under F , because

$$\|F(x) - E\|_{w,\infty} = \|F(x) - F(E)\|_{w,\infty} \leq r \|x - E\|_{w,\infty}.$$

Banach's fixed-point theorem therefore applies locally.

For the Π -map, fix a clade γ , and suppose all proper subclades have already been computed. Then the fixed-point map has the affine form

$$\Phi_\gamma(z) = C_\gamma + Lz,$$

where $z_e = \Pi_{e,\gamma}$, $C_\gamma \geq 0$ contains only already-known lower-clade terms, and the same linear operator $L = DF(E)$ appears on the same-clade terms SL_γ , DL_γ , and TL_γ . Therefore

$$\|\Phi_\gamma(z) - \Phi_\gamma(z')\|_{w,\infty} = \|L(z - z')\|_{w,\infty} \leq c\|z - z'\|_{w,\infty}.$$

So each bottom-up $\Pi_{\cdot,\gamma}$ map is not merely locally contracting : it is globally contracting in the same weighted norm.

If instead one views all $\Pi_{\cdot,\gamma}$ values for all clades as one large simultaneous fixed-point map, then the derivative at the fixed point is block triangular when clades are ordered by size. Each diagonal block is the same operator L . Hence all eigenvalues of the full derivative come from L , so its spectral radius is smaller than 1. In finite dimension, this implies that there exists an equivalent norm in which the full derivative has operator norm smaller than 1 ; by continuity, the simultaneous Π -map is also contracting in a sufficiently small neighbourhood of the fixed point.

In summary, with $p_l^{\text{obs}} = 1$ and $p_e^S > 0$ for all e , the E -map is locally contracting at E . Each bottom-up $\Pi_{\cdot,\gamma}$ map is globally contracting in the same weighted norm. The full interior assumption on $p_e^S, p_e^D, p_e^T, p_e^L$ is sufficient, but stronger than necessary.